

The Neural (MLP) Language Model

More context, learned embeddings, and a hidden layer — the Bengio (2003) architecture, with the exact shapes and math at every step. After Andrej Karpathy's makemore.

The bigram model could see only **one** previous character. To do better we want more context — but a count table over k previous characters has 27^k rows and becomes hopeless almost immediately. Bengio's 2003 paper showed the fix: learn a dense **embedding** vector for each character and feed a window of them through a small **multi-layer perceptron (MLP)**. Similar characters get similar embeddings, so the model generalizes across contexts it never literally saw.

What stays the same as Part 1

The objective is unchanged: produce logits, apply **softmax**, minimize the average negative log-likelihood (**cross-entropy**), and learn by backprop. We only enrich the **front** of the network — embeddings plus a hidden layer.

1. The context window

We fix a context length **block_size = 3**: predict each character from the three preceding ones. A name is scanned left to right, the context starting as three boundary tokens and sliding forward. For **emma**:

... -> e ..e -> m .em -> m emm -> a mma -> .

Each example is a length-3 vector of character indices (**X**) and the next index (**Y**). Across the dataset this yields about 228k examples, split **80/10/10** into train / dev / test.

Why three splits

Train fits the weights. **Dev** tunes the choices we make by hand (learning rate, sizes, how long to train). **Test** is touched once at the end for an honest final number. The gap between train and dev loss is the key diagnostic: close = underfitting (grow the model); train far below dev = overfitting (memorizing).

2. Embeddings: each character becomes a vector

Instead of a 27-long one-hot, every character gets a short, dense, **learnable** vector. We hold them in a table **C** of shape (27, 10) — row i is the 10-dimensional embedding of character i . Looking up the three context characters gives, per example, a (3, 10) block.

Crucially, **lookup is the same as a one-hot matmul**: $C[i]$ equals (one-hot of i) $\cdot C$, because a one-hot row selects one row of C . Indexing is just the fast way to do it.

$$\text{emb} = [C[c_1], C[c_2], C[c_3]] \in \mathbb{R}^{3 \times 10}$$

Why dense vectors generalize

The network can place **similar characters near each other** (the vowels, say). A pattern learned for one context then transfers to similar ones the model never saw — the generalization a sparse count table cannot get. This is the whole reason embeddings beat bigger n-gram tables.

3. The hidden layer: concatenate, then a non-linearity

The three 10-dim embeddings are **concatenated** into one 30-number vector, then passed through a fully-connected layer with a **tanh** non-linearity:

$$h = \tanh(\text{emb} \cdot W_1 + b_1), \quad W_1 \in \mathbb{R}^{30 \times 200}, \quad b_1 \in \mathbb{R}^{200}$$

- `emb.view(-1, 30)` does the concatenation for free — the (N, 3, 10) tensor is reinterpreted as (N, 30) with no copy; the -1 infers the batch size.
- **tanh** squashes each of the 200 hidden units into (-1, 1) and is what makes the model **non-linear** — able to represent interactions between context characters that the bigram's single matrix never could (e.g. 'q' is almost always followed by 'u').

4. The output layer: logits over 27 characters

A second linear layer maps the 200 hidden features to **27 logits** — one raw score per possible next character:

$$\text{logits} = h W_2 + b_2, \quad W_2 \in \mathbb{R}^{200 \times 27}, \quad b_2 \in \mathbb{R}^{27}$$

5. Softmax + cross-entropy, done stably

Softmax turns logits into probabilities, and the loss is the average negative log-likelihood of the true next character — identical in spirit to Part 1:

$$p_j = \frac{e^{\text{logit}_j}}{\sum_k e^{\text{logit}_k}}, \quad \text{loss} = -\frac{1}{N} \sum_t \log p_{t, y_t}$$

Why we do not hand-roll exp / normalize / log

If a logit is large, e^{logit} **overflows** to infinity. The fix is to subtract the per-row maximum logit before exponentiating — softmax is unchanged by that shift, but every exponent becomes ≤ 0 and safe. `F.cross_entropy` does exactly this, **fuses** log-softmax with the negative log-likelihood into one numerically stable kernel, and has a simpler, cheaper backward pass. Always use it.

6. The parameters

The learnable tensors are the embedding table and the two layers' weights and biases. The total here is about **11,900** numbers — tiny by modern standards, yet already better than the bigram model:

$$27 \times 10 + 30 \times 200 + 200 + 200 \times 27 + 27 = 11,897$$

the five terms are, in order: C (embedding table), W_{emb} , b_{emb} , W_{hid} , b_{hid} .

7. Training: learning rate, minibatches, decay

Two practical choices make training fast and stable:

- **Minibatches.** Each step uses a random batch of ~32 examples instead of all ~180k. The gradient is noisier but thousands of times cheaper, so we take far more steps — a much better use of compute. This is **stochastic** gradient descent.
- **Learning-rate search.** Sweep the step size exponentially from 10^{-3} to 10^0 and watch where the loss falls fastest; the good region here is around **0.1**.
- **Decay.** Start at 0.1 to move quickly, then drop to 0.01 to settle into the minimum without bouncing.

```
for i in range(steps):
    ix = torch.randint(0, Xtr.shape[0], (32,)) # minibatch
    emb = C[Xtr[ix]] # (32, 3, 10)
    h = torch.tanh(emb.view(-1, 30) @ W1 + b1) # (32, 200)
    logits = h @ W2 + b2 # (32, 27)
    loss = F.cross_entropy(logits, Ytr[ix])
    for p in parameters: p.grad = None
    loss.backward()
    lr = 0.1 if i < 0.6*steps else 0.01
    for p in parameters: p.data += -lr * p.grad
```

8. Does it generalize? Train vs dev loss

Evaluated without minibatch noise on the full splits, this model reaches roughly:

$$\text{train} \approx 2.19 \quad \text{dev} \approx 2.21 \quad (\text{bigram} \approx 2.45)$$

The two numbers being close means it is **generalizing**, not memorizing; both being below 2.45 means the extra context and the hidden layer genuinely helped.

9. The embedding space learns structure

The table C started as random noise; training shaped it. Plotting a 2-dimensional version of the embeddings shows meaningful geometry — notably the **vowels cluster together**, because they are interchangeable in many contexts, so placing them nearby lets the model reuse what it learns. That clustering is the generalization Bengio’s design buys.

10. Generating names

Generation mirrors the bigram model, but the distribution now comes from the full MLP: start with three boundary tokens, run the forward pass, sample the next character, slide it into the context (dropping the oldest), and repeat until . is drawn. Because it uses three characters of context and shares strength through the embeddings, the samples are noticeably more name-like — e.g. *kayden*, *marian*, *deliah*, *quinn*.

11. Summary: what the MLP added

	Bigram (Part 1)	MLP (Part 2)
Context	1 previous char	3 previous chars
Representation	one-hot (sparse, 27-dim)	learned embedding (dense, 10-dim)
Model	one linear layer + softmax	embed → concat → tanh → linear → softmax
Scaling context	table size 27^k — explodes	input grows a little; params barely change
Loss (this run)	~2.45	~2.2 (lower is better)

The throughline: the loss, the softmax, the cross-entropy, and the backprop loop are **unchanged** from Part 1 — we only gave the network more context and a richer way to represent it. That is the entire recipe for scaling a language model, and the next parts of the series (deeper nets, better initialization and normalization, and eventually attention) push exactly this further.

Companion: the runnable notebook **MLP_explained.ipynb** and the interactive visualizer **MLP, Animated** (th3samaritan.github.io/Al-Research/mlp).